المحاضرة (2)

تصميم البرمجيات

(Software Design)

The Institute of Electrical and Electronics Engineers, Inc.
345East 47th Street, New York, NY 10017-2394, USA

- **مقدمة.**
  - ❖ **عملية تصميم البرمجيات:**
  - ❖ **الأنماط المعمارية:**
  - ❖ **خصائص التصميم:**
  - ❖ **عناصر عملية تصميم البرمجيات:**
  - ❖ **تقنيات التمكين:**

- **مقدمة:**

يلعب تصميم البرمجيات دورًا مهمًا في تطوير البرمجيات: فهو يسمح لمهندسي البرمجيات بإنتاج نماذج مختلفة تشكل نوعًا من مخطط الحل الذي سيتم تنفيذه. يمكننا تحليل وتقييم هذه النماذج لتحديد ما إذا كانت ستسمح لنا بتلبية المتطلبات المختلفة أم لا. يمكننا أيضًا فحص وتقييم مختلف الحلول البديلة والمقايضات. وأخيرا، يمكننا استخدام النماذج الناتجة لتخطيط أنشطة التطوير اللاحقة، بالإضافة إلى استخدامها كمدخلات ونقطة انطلاق للبناء والاختبار.

* يجب علينا أن نكون على معرفة جيدة بأساسيات تصميم البرامج وذلك لكي تتمكن من:

- التعرف على المفاهيم الأساسية المطلوبة لتصميم البرمجيات.
- وصف مشكلة التصميم من خلال عناصرها الأساسية.
- تحديد تقنيات تمكين التصميم وشرح فوائدها.
- تمييز أنشطة التصميم منخفضة المستوى عن أنشطة التصميم عالية المستوى ومتى يجب تطبيق أي منهما.

عملية تصميم البرمجيات:

نظرة عامة حول عملية تصميم البرمجيات، بدءًا من فهم المتطلبات وصولًا إلى إنشاء تصميم مفصل وقابل للتنفيذ.

- المراحل الرئيسية لعملية التصميم، والاعتبارات الهامة، وأفضل الممارسات لضمان تطوير برمجيات عالية الجودة تلبي احتياجات المستخدمين:

1. فهم المتطلبات

تعتبر مرحلة فهم المتطلبات حجر الأساس في عملية تصميم البرمجيات. قبل البدء في أي تصميم، من الضروري فهم ما هو المطلوب من البرنامج، وما هي المشاكل التي يهدف إلى حلها، وما هي توقعات المستخدمين.

ويمكن تحقيق ذلك من خلال:

- **جمع المتطلبات:** يتضمن ذلك جمع المعلومات من المستخدمين وأصحاب المصلحة من خلال المقابلات والاستبيانات وورش العمل.
- **تحليل المتطلبات:** يتم تحليل المتطلبات المجمعة لتحديد: المتطلبات الوظيفية (ماذا يجب أن يفعله البرنامج) وغير الوظيفية (مثل الأداء والأمان وقابلية الاستخدام).
- **توثيق المتطلبات:** يتم توثيق المتطلبات بشكل واضح ومفصل في وثيقة متطلبات البرمجيات (SRS).

2. التصميم عالي المستوى (التصميم المعماري)

بعد فهم المتطلبات، تبدأ مرحلة التصميم عالي المستوى، والتي تركز على تحديد البنية العامة للبرنامج. يتضمن ذلك:

- **تحديد الوحدات النمطية (Modules):** تقسيم البرنامج إلى وحدات نمطية أصغر وأكثر قابلية للإدارة.
- **تحديد الواجهات (Interfaces):** تحديد كيفية تفاعل الوحدات النمطية مع بعضها البعض.
- **تحديد التقنيات:** اختيار التقنيات والأدوات التي سيتم استخدامها في تطوير البرنامج.
- **اختيار الأنماط المعمارية (Architectural Patterns):** اختيار نمط معماري مناسب، مثل نمط MVC (Model-View-Controller) أو نمط طبقات (Layered Architecture).

3. التصميم منخفض المستوى (التصميم التفصيلي)

بعد الانتهاء من التصميم عالي المستوى، تبدأ مرحلة التصميم منخفض المستوى، والتي تركز على تفاصيل تنفيذ الوحدات النمطية. يتضمن ذلك:

- **تصميم الخوارزميات:** تحديد الخوارزميات التي سيتم استخدامها في كل وحدة نمطية.
- **تصميم هياكل البيانات:** تحديد هياكل البيانات التي سيتم استخدامها لتخزين البيانات.
- **تصميم الواجهات:** تحديد تفاصيل الواجهات بين الوحدات النمطية.
- **تصميم قواعد البيانات:** تصميم قواعد البيانات التي سيتم استخدامها لتخزين البيانات الدائمة.

4. اعتبارات هامة في عملية التصميم

هناك العديد من الاعتبارات الهامة التي يجب أخذها في الاعتبار أثناء عملية تصميم البرمجيات، منها:

- **قابلية التوسع (Scalability):** يجب تصميم البرنامج بحيث يكون قابلاً للتوسع للتعامل مع زيادة عدد المستخدمين أو البيانات.
- **الأداء (Performance):** يجب تصميم البرنامج بحيث يكون سريعًا وفعالًا.
- **الأمان (Security):** يجب تصميم البرنامج بحيث يكون آمنًا ضد الهجمات الإلكترونية.
- **قابلية الصيانة (Maintainability):** يجب تصميم البرنامج بحيث يكون سهل الصيانة والتعديل.
- **قابلية الاستخدام (Usability):** يجب تصميم البرنامج بحيث يكون سهل الاستخدام للمستخدمين.

5. أفضل الممارسات في عملية التصميم

من الممارسات التي يمكن اتباعها لضمان تطوير برمجيات عالية الجودة:

- **استخدام مبادئ التصميم الجيد:** تسمى أيضاً **(تقنيات التمكين)** وهي مفاهيم أساسية ومبادئ توجيهية للعديد من أساليب ومفاهيم تصميم البرمجيات المختلفة التي غالبا ما تنتج برمجيات جيدة وذات جودة عالية.
- **استخدام أنماط التصميم (Design Patterns):** وهي حلول مجربة ومختبرة لمشاكل التصميم الشائعة.
- **إجراء مراجعات التصميم (Design Reviews):** لمراجعة التصميم من قبل فريق من الخبراء.
- **كتابة وثائق تصميم واضحة ومفصلة:** لتسهيل فهم التصميم من قبل المطورين.
- **استخدام أدوات التصميم (Design Tools):** مثل أدوات UML (Unified Modeling Language) لنمذجة التصميم.
- **التصميم التكراري (Iterative Design):** تصميم البرنامج بشكل تكراري، مع الحصول على ملاحظات المستخدمين في كل تكرار.

6. أدوات التصميم

تتوفر العديد من الأدوات التي يمكن استخدامها في عملية تصميم البرمجيات، ومنها:

- **أدوات UML:** مثل Enterprise Architect و Visual Paradigm لنمذجة التصميم باستخدام UML.
- **أدوات النماذج الأولية (Prototyping Tools):** مثل Figma و Adobe XD لإنشاء نماذج أولية للواجهات المستخدم.
- **أدوات إدارة المتطلبات (Requirements Management Tools):** مثل Jira و Azure DevOps لإدارة المتطلبات وتتبعها.

الأنماط المعمارية (Architectural Patterns)

❖ نمط MVC (Model-View-Controller):

هو نمط تصميم يُستخدم في تطوير البرمجيات لفصل البيانات، واجهة المستخدم، وعناصر التحكم في التطبيق. يهدف هذا النمط إلى تعزيز التنظيم والمرونة وسهولة الصيانة.

- أقسامه :

النموذج (Model):

- يمثل البيانات والتطبيقات المنطقية.
- يتعامل مع قواعد البيانات ومعالجة البيانات.
- يُستخدم لتحديث البيانات وإخطار العناصر الأخرى بالتغييرات.

العرض (View):

- يمثل واجهة المستخدم.
- يُظهر البيانات للمستخدم ويستقبل المدخلات.
- يمكن أن يتغير دون التأثير على البيانات أو المنطق.

وحدة التحكم (Controller):

- يعمل كحلقة وصل بين النموذج والعرض.
- يتلقى المدخلات من المستخدم (من العرض) ويقوم بتحديث النموذج وفقًا لذلك.
- يُحدد كيف تتفاعل الأجزاء المختلفة للتطبيق.

- فوائد استخدام نمط MVC:

- فصل المسؤوليات: يسهل فهم وصيانة الكود.
- إعادة الاستخدام: يمكن استخدام النماذج والعروض في تطبيقات مختلفة.
- تسهيل الاختبار: يمكن اختبار كل عنصر بشكل مستقل.

❖ - نمط الطبقات (Layered Architecture):

هو نمط تصميم برمجي يُستخدم لفصل التطبيق إلى عدة طبقات، حيث كل طبقة تؤدي وظيفة معينة وتتعامل مع بيانات معينة. يهدف هذا النمط إلى تحسين تنظيم الكود، وتعزيز القابلية للصيانة، وإعادة الاستخدام.

- الطبقات الأساسية في نمط الطبقات:

1. طبقة العرض (Presentation Layer).
   - تتعامل مع واجهة المستخدم.
   - تعرض البيانات وتستقبل المدخلات من المستخدم.
   - قد تشمل عناصر واجهة المستخدم مثل الأزرار والنماذج.

2. طبقة الأعمال (Business Logic Layer):
   - تحتوي على منطق الأعمال وتطبيق القواعد.
   - تعالج المدخلات وتقوم بتنفيذ العمليات المطلوبة.
   - تعمل كحلقة وصل بين طبقة العرض وطبقة البيانات.

3. طبقة البيانات (Data Access Layer):
   - تتعامل مع التخزين واسترجاع البيانات.
   - تتفاعل مع قواعد البيانات أو أي مصادر بيانات أخرى.
   - قد تشمل عمليات مثل الاستعلامات والإدخالات.

- فوائد استخدام نمط الطبقات:

- فصل المسؤوليات: يسهل فهم وصيانة كل طبقة بشكل مستقل.
- تسهيل الاختبار: يمكن اختبار كل طبقة بمفردها، مما يسهل اكتشاف الأخطاء.
- إعادة الاستخدام: يمكن استخدام الطبقات في تطبيقات مختلفة أو استبدال طبقة بأخرى دون التأثير على باقي النظام.

خصائص التصميم

لماذا يجب علينا تصميم البرمجيات قبل البدء في إنشائها ؟

- **يتيح التصميم التقييم المبكر:** يمكن أن تكون عملية بناء برمجيات عالية الجودة مكلفة للغاية من حيث الوقت والمال، يوصى عند التصميم بالتقييم المبكرً قبل البدء من أجل توفير الوقت والمال.

- **متطلبات التصميم للنمذجة:** عند النمذجة ، ينصب تركيز المصمم على المشكلة ، مع ترك بعض التعقيد المتأصل في المنتج النهائي. نظرًا لأن التعقيد هو أحد أكبر الصعوبات في التطوير.

- **التصميم يتضمن التخطيط:** تساعد عملية التصميم في تقدير التكاليف ، وتحديد الجدول الزمني ، وكم عدد أفراد الطاقم التنفيذي ، وما الأجزاء الموجودة التي يمكن إعادة استخدامها في التصميم الحالي ، كما يتضمن التحكم في التطوير.

- **التصميم يسهل التواصل:** يوفر التصميم لغة مشتركة يمكن من خلالها التعريف بالمشروع ومناقشة المشاكل المتوقعة. لذا إذا كان المشروع يتضمن أكثر من شخص واحد فيجب أن يكون التواصل بين الطاقم التنفيذي شيء ضروري ومستمر .

مما سبق نستنتج أنه من فوائد التصميم إمكانية التقييم المبكر الذي يساعد على ضمان انتاج وتطوير البرنامج الصحيح الذي يلبي احتياجات العميل كما يساعد في تقدير التكاليف ، والوقت المستغرق لعملية التطوير ، وعدد الموظفين .... إلخ

عناصر عملية تصميم البرمجيات

يمكن وصف عملية التصميم بأنها عملية اختيار تمثيل للحل من بين مجموعة من البدائل ، بالنظر إلى القيود المفروضة على مجموعة من الأهداف. يمكن تقسيمها إلى مرحلتين هما (**التنويع - التقارب**).

- **التنويع:** هو مرحلة توليد البدائل ، ليس بالضرورة وثائق تصف حلًا ممكنًا ، ولكن ، على الأقل ، في ذهن المصمم. هذه البدائل هي الحلول المرشحة ويتم إنشاؤها و الحصول عليها من المعرفة أو الكتالوجات أو الخبرة السابقة.
- **التقارب**: خلال مرحلة التقارب يختار المصمم البديل (أو مجموعة البدائل) التي تلبي الأهداف المطلوبة. ويكون البديل المختار هو الحل ، الذي سيلبي القيود التي يفرضها مجال المشكلة.

ومن أهم العناصر التي تساعد على فهم عملية تصميم البرمجيات:

(الأهداف) (القيود) (البدائل) (التوكيلات أو التمثيلات) (والحلول)

- **الأهداف :**

يبدأ التصميم بالحاجة. في هندسة البرمجيات ، تبدأ الضرورة مع العميل الذي يحدد ما هي احتياجاته وبالتالي، فإن الهدف من عملية التصميم هو تحقيق حل يلبي احتياجات العميل. في تصميم البرمجيات، يشار إلى الأهداف أيضًا بالمتطلبات. يهتم تصميم البرامج بشكل أساسي بنوعين من المتطلبات

1- **المتطلبات الوظيفية:** تحدد الوظيفة التي سيظهرها النظام، ويجب أن تكون مرضية لمتطلبات العميل.
2- **المتطلبات غير الوظيفية:** تحدد الخصائص التي يجب أن تلتزم بها البرمجيات والتي لا تتعلق بالوظائف نفسها. ومن هذه المتطلبات (الأداء – الأمان – القابلية للصيانة – القابلية للتوسع – قابلية النقل).

- **القيود:**

يجب أن يكون منتج التصميم ممكنًا. بالنظر إلى ذلك ، فإن قيد التصميم هو قاعدة أو شرط أو علاقة أو اتفاقية أو مبدأ يحدد سياق التصميم ، من أجل تحقيق منتج تصميم عملي.
من المهم معرفة أن القيود مرتبطة بالأهداف. ومع ذلك ، من أجل التفريق بينهما ، من المهم أن نفهم أن الأهداف ليست هي التي تتحكم فيما سيتم تصميمه. بمعنى آخر ، قد يكون لنظام البرمجيات أهداف واضحة و لكن تصميمه أو بعض القيود المتعلقة به ، قد لا تجعله ممكنًا.

- **البدائل:**

بديل التصميم هو إمكانية الحل. نظرًا لأن مشكلات التصميم غالبًا ما يكون لها حلول متعددة فمن المتوقع أن ينشئ المصمم بدائل تصميم متعددة لمشكلة واحدة. يجب أن نفهم أن المصمم بعد فهم أهداف المشكلة وقيودها ، لديه شاغلان: الجيل البديل واختيار الحل من بين البدائل .

- **التوكيلات:**

التوكيل (التمثيل) إن اشراك أصحاب المصلحة أو من يمثلهم (الوكيل) في العمل المشترك أو بالاتصال مع فريق العمل يدعم عملية التصميم ويساعد في سرعة انجاز العمل وتفادي الانحراف عن الأهداف المطلوبة.

- **الحلول:**

يقصد بها الاستراتيجيات أو الطرق التي يتم اعتمادها لمعالجة المشاكل أو التحديات التي قد تواجه فريق التطوير، وهذه الحلول تهدف إلى تحسين جودة البرمجيات وزيادة كفاءتها وتقليل التكاليف والمخاطر المرتبطة بعملية التطوير .

تقنيات التمكين

تقنيات التمكين الأساسية للمصمم الناجح هي:

- **التجريد (التلخيص).**
- **إخفاء المعلومات.**
- **النمذجة.**
- **فصل المخاوف.**
- **الاقتران والتماسك.**
- **فصل السياسة عن التنفيذ.**
- **فصل الواجهة عن التنفيذ.**